CamConnect State Management Architecture

Overview

The CamConnect application implements a sophisticated state management architecture that ensures predictable data flow, efficient updates, and maintainable code structure. This document outlines the state management patterns, ViewModels, data models, and architectural decisions.

Architecture Principles

1. Unidirectional Data Flow

The application follows a unidirectional data flow pattern where: - **State flows down**: Parent components pass state to child components - **Events flow up**: Child components emit events that bubble up to state holders - **Single source of truth**: Each piece of state has a single authoritative source

2. Separation of Concerns

- **UI State**: Managed by Compose state and ViewModels
- **Business Logic**: Encapsulated in ViewModels and use cases
- **Data Layer**: Handled by repositories and data sources
- **Presentation**: Pure UI components without business logic

3. Reactive Programming

- **StateFlow**: For reactive state management
- **Coroutines**: For asynchronous operations
- **Lifecycle Awareness**: Proper lifecycle management with collectAsStateWithLifecycle

ViewModel Architecture

RecordingViewModel

Manages recording state and functionality with comprehensive state handling.

State Management

```
class RecordingViewModel : ViewModel() {
    // Private mutable state flows
    private val _isRecording = MutableStateFlow(false)
    private val _recordingState = MutableStateFlow<RecordingState>(RecordingState.NotRecording)
    private val _recordingDuration = MutableStateFlow("00:00")

    // Public immutable state flows
    val isRecording: StateFlow<Boolean> = _isRecording.asStateFlow()
    val recordingState: StateFlow<RecordingState> = _recordingState.asStateFlow()
    val recordingDuration: StateFlow<String> = _recordingDuration.asStateFlow()

    // Recording timer job
    private var recordingTimerJob: Job? = null
}
```

Recording States

```
sealed class RecordingState {
    object NotRecording : RecordingState()
    data class Recording(val duration: String) : RecordingState()
```

```
    object StoppingRecording : RecordingState()
    object SavedToGallery : RecordingState()
    data class Error(val message: String) : RecordingState()
}
```

Recording Logic

```
fun toggleRecording(context: Context) {
    viewModelScope.launch {
        when (recordingState.value) {
            is RecordingState.NotRecording -> startRecording(context)
            is RecordingState.Recording -> stopRecording()
            else -> { /* Handle other states */ }
        }
    }
}

private suspend fun startRecording(context: Context) {
    try {
        _recordingState.value = RecordingState.Recording("00:00")
        _isRecording.value = true

        // Start recording timer
        recordingTimerJob = viewModelScope.launch {
            var seconds = 0
            while (isRecording.value) {
                delay(1000)
                seconds++
                _recordingDuration.value = formatDuration(seconds)
                _recordingState.value = RecordingState.Recording(_recordingDuration.value)
            }
        }

        // Initialize recording service
        initializeRecordingService(context)

    } catch (e: Exception) {
        _recordingState.value = RecordingState.Error(e.message ?: "Recording failed")
        _isRecording.value = false
    }
}

private suspend fun stopRecording() {
    _recordingState.value = RecordingState.StoppingRecording

    try {
        // Stop recording service
        stopRecordingService()

        // Stop timer
        recordingTimerJob?.cancel()

        _isRecording.value = false
        _recordingState.value = RecordingState.SavedToGallery
```

```
        // Reset after showing success state
        delay(2000)
        _recordingState.value = RecordingState.NotRecording

    } catch (e: Exception) {
        _recordingState.value = RecordingState.Error(e.message ?: "Failed to stop recording")
        _isRecording.value = false
    }
}
```

CameraControlViewModel

Manages camera control states and operations with real-time updates.

State Structure

```
data class CameraControlState(
    val currentZoom: Float = 1f,
    val isZoomEnabled: Boolean = true,
    val isIrEnabled: Boolean = false,
    val isLowIntensity: Boolean = true,
    val isEisEnabled: Boolean = false,
    val isHdrEnabled: Boolean = false,
    val isAutoDayNightEnabled: Boolean = false,
    val visionMode: VisionMode = VisionMode.VISION,
    val errorMessage: String? = null
)
```

Implementation

```
class CameraControlViewModel : ViewModel() {
    private val _cameraControlState = MutableStateFlow(CameraControlState())
    val cameraControlState: StateFlow<CameraControlState> = _cameraControlState.asStateFlow()

    // API helper for camera operations
    private val motocamAPIHelper = MotocamAPIAndroidHelper()

    fun setZoom(zoom: Float) {
        viewModelScope.launch {
            if (!_cameraControlState.value.isZoomEnabled) return@launch

            val zoomLevel = when (zoom) {
                1f -> MotocamAPIHelper.ZOOM.X1
                2f -> MotocamAPIHelper.ZOOM.X2
                else -> MotocamAPIHelper.ZOOM.X4
            }

            motocamAPIHelper.setZoomAsync(scope = viewModelScope, zoom = zoomLevel) { result, error ->
                if (error == null && result) {
                    _cameraControlState.value = _cameraControlState.value.copy(currentZoom = zoom)
                } else {
                    _cameraControlState.value = _cameraControlState.value.copy(
                        errorMessage = error?.message ?: "Zoom operation failed"
                    )
```

```
                }
            }
        }
    }

    fun toggleIR() {
        viewModelScope.launch {
            val newIrState = !_cameraControlState.value.isIrEnabled
            _cameraControlState.value = _cameraControlState.value.copy(isIrEnabled = newIrState)

            // API call to update IR state
            motocamAPIHelper.setIRAsync(scope = viewModelScope, enabled = newIrState) { result, error ->
                if (error != null) {
                    // Revert state on error
                    _cameraControlState.value = _cameraControlState.value.copy(isIrEnabled = !newIrState)
                }
            }
        }
    }

    fun toggleIrIntensity() {
        viewModelScope.launch {
            val newIntensity = !_cameraControlState.value.isLowIntensity
            _cameraControlState.value = _cameraControlState.value.copy(isLowIntensity = newIntensity)

            // API call to update IR intensity
            motocamAPIHelper.setIrIntensityAsync(scope = viewModelScope, lowIntensity = newIntensity) {
                if (error != null) {
                    // Revert state on error
                    _cameraControlState.value = _cameraControlState.value.copy(isLowIntensity = !newInte
                }
            }
        }
    }

    fun refreshCameraState() {
        viewModelScope.launch {
            // Refresh camera state from API
            motocamAPIHelper.getCameraStateAsync(scope = viewModelScope) { state, error ->
                if (error == null && state != null) {
                    _cameraControlState.value = state
                }
            }
        }
    }
}
```

CameraLayoutViewModel

Manages layout-specific camera states and vision modes.

Implementation

```
class CameraLayoutViewModel : ViewModel() {
    private val _currentVisionMode = MutableStateFlow(VisionMode.VISION)
```

```
    val currentVisionMode: StateFlow<VisionMode> = _currentVisionMode.asStateFlow()

    private val _layoutState = MutableStateFlow(LayoutMode.MINIMAL_CONTROL)
    val layoutState: StateFlow<LayoutMode> = _layoutState.asStateFlow()

    fun setVisionMode(mode: VisionMode) {
        viewModelScope.launch {
            _currentVisionMode.value = mode

            // Update camera configuration based on vision mode
            when (mode) {
                VisionMode.VISION -> {
                    // Standard vision mode configuration
                }
                VisionMode.INFRARED -> {
                    // Infrared mode configuration
                }
                VisionMode.BOTH -> {
                    // Dual mode configuration
                }
            }
        }
    }

    fun setLayoutMode(mode: LayoutMode) {
        _layoutState.value = mode
    }
}
```

Data Models

CameraState

Comprehensive camera state representation.

```
data class CameraState(
    val isRecording: Boolean = false,
    val currentCamera: Int = 0,
    val zoomLevel: Float = 1f,
    val cameraMode: CameraMode = CameraMode.NORMAL,
    val visionMode: VisionMode = VisionMode.VISION,
    val orientation: Orientation = Orientation.PORTRAIT,
    val autoDayNight: Boolean = false,
    val hasError: Boolean = false,
    val errorMessage: String? = null
)
```

SystemStatus

System-wide status monitoring.

```
data class SystemStatus(
    val batteryLevel: Int = 100,
    val isWifiConnected: Boolean = false,
    val isLteConnected: Boolean = false,
    val isOnline: Boolean = false,
```

```
    val isAiEnabled: Boolean = false,
    val currentSpeed: Float = 0f,
    val compassDirection: Float = 0f,
    val lastUpdated: Long = System.currentTimeMillis()
)
```

DetectionSettings

AI detection configuration.

```
data class DetectionSettings(
    val objectDetection: Boolean = false,
    val farObjectDetection: Boolean = false,
    val motionDetection: Boolean = false,
    val detectionSensitivity: Float = 0.5f
)
```

State Flow Patterns

1. State Observation

Using `collectAsStateWithLifecycle` for lifecycle-aware state collection.

```
@Composable
fun CameraControlScreen() {
    val recordingViewModel: RecordingViewModel = viewModel()
    val cameraControlViewModel: CameraControlViewModel = viewModel()

    val isRecording by recordingViewModel.isRecording.collectAsStateWithLifecycle()
    val recordingState by recordingViewModel.recordingState.collectAsStateWithLifecycle()
    val cameraControlState by cameraControlViewModel.cameraControlState.collectAsStateWithLifecycle()

    // UI implementation using observed states
}
```

2. State Hoisting

Lifting state up to parent components for shared state management.

```
@Composable
fun ParentComponent() {
    var sharedState by remember { mutableStateOf(InitialState) }

    ChildComponent(
        state = sharedState,
        onStateChange = { newState -> sharedState = newState }
    )
}
```

3. Derived State

Computing derived state from primary state sources.

```
@Composable
fun ComponentWithDerivedState(primaryState: PrimaryState) {
    val derivedState by remember(primaryState) {
        derivedStateOf {
```

```
            // Compute derived state from primary state
            computeDerivedState(primaryState)
        }
    }
}
```

Memory Management

1. DisposableEffect Usage

Proper cleanup of resources and listeners.

```
@Composable
fun ComponentWithResources() {
    val context = LocalContext.current

    DisposableEffect(context) {
        val receiver = object : BroadcastReceiver() {
            override fun onReceive(context: Context?, intent: Intent?) {
                // Handle broadcast
            }
        }

        val filter = IntentFilter(Intent.ACTION_BATTERY_CHANGED)
        context.registerReceiver(receiver, filter)

        onDispose {
            context.unregisterReceiver(receiver)
        }
    }
}
```

2. ViewModel Cleanup

Proper cleanup in ViewModels.

```
class CleanupViewModel : ViewModel() {
    private var cleanupJob: Job? = null

    override fun onCleared() {
        super.onCleared()
        cleanupJob?.cancel()
        // Additional cleanup
    }
}
```

3. Memory Manager Integration

Using the application's memory management system.

```
DisposableEffect(Unit) {
    Log.d("Component", "Component created")
    onDispose {
        Log.d("Component", "Component disposed - cleaning up")
        try {
            MemoryManager.cleanupWeakReferences()
        } catch (e: Exception) {
```

```
            Log.e("Component", "Error during cleanup", e)
        }
    }
}
```

Error Handling

1. State-Based Error Handling

Incorporating error states into data models.

```
sealed class Result<out T> {
    data class Success<T>(val data: T) : Result<T>()
    data class Error(val message: String, val exception: Exception? = null) : Result<Nothing>()
    object Loading : Result<Nothing>()
}
```

2. Error Recovery

Implementing error recovery mechanisms.

```
fun performOperation() {
    viewModelScope.launch {
        try {
            _state.value = Result.Loading
            val result = apiCall()
            _state.value = Result.Success(result)
        } catch (e: Exception) {
            _state.value = Result.Error(e.message ?: "Operation failed", e)

            // Attempt recovery
            delay(5000)
            performOperation() // Retry
        }
    }
}
```

Performance Optimization

1. State Flow Optimization

Using efficient state flow patterns.

```
// Use distinctUntilChanged for expensive computations
val expensiveState by expensiveComputation
    .distinctUntilChanged()
    .collectAsStateWithLifecycle()
```

2. Lazy State Initialization

Deferring expensive state initialization.

```
val lazyState by remember {
    derivedStateOf {
        // Expensive computation only when dependencies change
        computeExpensiveState(dependencies)
    }
}
```

3. State Caching

Caching frequently accessed state.

```
private val stateCache = mutableMapOf<String, Any>()

fun getCachedState(key: String): Any? {
    return stateCache[key] ?: run {
        val computedState = computeState(key)
        stateCache[key] = computedState
        computedState
    }
}
```

Testing State Management

1. ViewModel Testing

Testing ViewModels with proper state verification.

```
@Test
fun `test recording state transitions`() = runTest {
    val viewModel = RecordingViewModel()

    // Initial state
    assertEquals(RecordingState.NotRecording, viewModel.recordingState.value)

    // Start recording
    viewModel.toggleRecording(mockContext)
    assertEquals(RecordingState.Recording("00:00"), viewModel.recordingState.value)

    // Stop recording
    viewModel.toggleRecording(mockContext)
    assertEquals(RecordingState.SavedToGallery, viewModel.recordingState.value)
}
```

2. State Flow Testing

Testing state flows with Turbine.

```
@Test
fun `test state flow emissions`() = runTest {
    val viewModel = RecordingViewModel()

    viewModel.isRecording.test {
        assertEquals(false, awaitItem()) // Initial state

        viewModel.toggleRecording(mockContext)
        assertEquals(true, awaitItem()) // Recording started
    }
}
```

Best Practices

1. State Design

- **Immutable State**: All state should be immutable

- **Single Source of Truth**: Each piece of state has one authoritative source
- **Minimal State**: Only store essential state, compute derived values
- **Type Safety**: Use sealed classes for complex state hierarchies

2. State Updates

- **Atomic Updates**: Update related state atomically
- **Predictable Updates**: State updates should be predictable and testable
- **Error Handling**: Always handle error states in state updates
- **Performance**: Use efficient state update patterns

3. State Observation

- **Lifecycle Awareness**: Use lifecycle-aware state collection
- **Efficient Observation**: Only observe necessary state changes
- **Memory Management**: Properly dispose of state observers
- **Error Recovery**: Implement proper error recovery mechanisms

Future Enhancements

1. State Persistence

- **Room Database**: Persistent state storage
- **DataStore**: Preferences and settings persistence
- **State Restoration**: App state restoration after process death

2. Advanced State Management

- **Redux Pattern**: More complex state management patterns
- **State Machines**: Finite state machines for complex workflows
- **Event Sourcing**: Event-driven state management

3. Performance Improvements

- **State Compression**: Efficient state serialization
- **Lazy Loading**: On-demand state loading
- **Caching Strategies**: Advanced caching mechanisms

4. Testing Enhancements

- **State Snapshot Testing**: Visual state testing
- **Integration Testing**: End-to-end state testing
- **Performance Testing**: State management performance testing

---

Document Version: 1.0
Last Updated: 15-07-2025
Author: Shathir
Status: Complete **License**: Apache 2.0